MCLite (MeshCore) — Complete Hardware Guide

> **Firmware:** MCLite (MeshCore companion firmware) · **Upstream:** laserir/MCLite (MIT) **Chip:** ESP32-S3 · **Radio:** Semtech SX1262 LoRa · **Boards:** LilyGo T-Deck Plus, LilyGo T-Watch Ultra **Cyber Controller profile:** `mclite` (esptool backend, merged single-bin @ `0x0`, esp32s3, 115200 baud, not suicide-capable) **This guide:** purchase → build → flash → integrate into Cyber Controller → use → troubleshoot.

1. Overview

MCLite is a lightweight **off-grid / emergency communicator** firmware built on the **MeshCore** mesh protocol, purpose-made for two LilyGo handhelds: the **T-Deck Plus** and the **T-Watch Ultra**. It turns an ESP32-S3 + SX1262 LoRa board into a self-contained encrypted messenger that needs no internet, cell towers, accounts, or pairing — useful for families, hiking/camping groups, search-and-rescue, events, and disaster comms. Core features are encrypted **direct messages**, public/private/hashtag **channels**, **room servers** (store-and-forward bulletin boards), manual **GPS location sharing** with privacy-grid coarsening, **telemetry** requests (battery/position), an offline **map view**, and an **SOS** alert.

MeshCore (and therefore MCLite) is *not* Meshtastic. Both run on similar LoRa hardware, but the routing model differs: **Meshtastic** uses managed flood routing where every client can relay; **MeshCore** uses structured path routing where only dedicated **Repeater** nodes forward traffic and end-user **Companion** nodes do not rebroadcast. MeshCore supports far longer relay chains (advertised up to 64 hops) and adds **Room Servers** for offline message retrieval — features Meshtastic lacks. MCLite "speaks the standard MeshCore companion protocol," so an MCLite device interoperates with other MeshCore nodes and the official MeshCore iOS/Android apps on the same network.

Cyber Controller's role here is **flashing and serial access**: it writes the correct merged `.bin` to the board and exposes the USB serial console. Day-to-day mesh use happens on the device's own UI or via companion mode (see §6–§7).

2. Legal & Safety

MCLite is **lawful mesh-networking firmware** (MIT licensed). There are no offensive/attack functions and the `mclite` profile is **not suicide-capable** — Cyber Controller marks it non-destructive. The legal considerations are **radio-regulatory, not computer-crime**:

- **Use a licence-free ISM band for your region.** LoRa here runs in regional ISM/SRD bands — **868 MHz (EU/UK)** and **915 MHz (US/Canada/Australia/NZ)**. Transmitting on the wrong band, or above the legal ERP/power, can be illegal. Pick the matching MCLite **region preset** (see §3) so frequency and power stay compliant.
- **Duty cycle / airtime. EU/UK/CH:** ETSI **EN 300 220** enforces a **10% TX duty cycle** in the relevant sub-band — MCLite applies a firmware-level 10% cap as a *best-effort* aid. **US/Canada:** FCC **Part 15.247** has no duty-cycle limit (MCLite uses a ~33% airtime budget). **Australia:** ACMA class licence **LIPD-2015** (~33% airtime budget). *The firmware cap is an aid, not a compliance guarantee — the operator is responsible.* (verify exact current limits for your country before deploying.)
- **Antenna matters.** Always have the LoRa antenna connected before transmitting; running the SX1262 PA into no/poor antenna risks damaging the radio.
- **Encryption is built in** (Ed25519 keys; the private key never leaves the device). This is normal, lawful end-to-end protection — but check that strong encryption on radio is permitted where you operate.

3. Hardware & Purchasing

MCLite supports exactly **two boards** (per the `mclite` profile). Both are ESP32-S3 with 16 MB flash + 8 MB PSRAM and an **SX1262** LoRa radio:

Board	Form factor	Key hardware	Buy (search terms)
LilyGo T-Deck Plus	BlackBerry-style handheld	ESP32-S3 (16 MB flash), 2.8" ST7789 320×240 IPS, **QWERTY keyboard + trackball**, internal **GPS**, SX1262 LoRa w/ **SMA external antenna**, 2000 mAh battery, microSD	LilyGo store: "**T-Deck Plus**"; also Amazon / Micro Center / passion-radio "LILYGO T-Deck Plus SX1262"
LilyGo T-Watch Ultra	Wrist smartwatch	ESP32-S3 (16 MB flash), **2.06" AMOLED touch 410×502**, **u-blox MIA-M10Q GNSS**, SX1262 LoRa, 1100 mAh, IP65, mic/NFC/haptic, microSD	LilyGo store: "**T-Watch Ultra**"; also Amazon / Newegg / OpenELAB "LILYGO T-Watch Ultra LoRa"

Buy the right frequency variant. LilyGo sells these boards in **per-band** versions (e.g. 433 / 868 / 915 MHz). Order the version matching your region's ISM band (EU/UK → **868 MHz**; US/AU/NZ → **915 MHz**) — the SX1262 chip is wideband but the **onboard matching/antenna are tuned per band**, so the wrong variant transmits poorly. (verify the exact SKU band before checkout.)

Accessories: - A **data-capable USB-C cable** (not charge-only) — required for flashing. - A **microSD card, FAT32, ≤ 32 GB** — MCLite stores `config.json`, message history, language packs, and optional offline **map tile packs** here. A blank card is enough for first boot (auto-config). - For the **T-Deck Plus**, a matching **SMA LoRa antenna** for your band (an SMA whip usually ships with it — verify it is the LoRa antenna, not a stray GPS/WiFi antenna, and that it matches your band). - Optional belt case / lanyard; the T-Deck Plus is often sold with an ABS shell + strap.

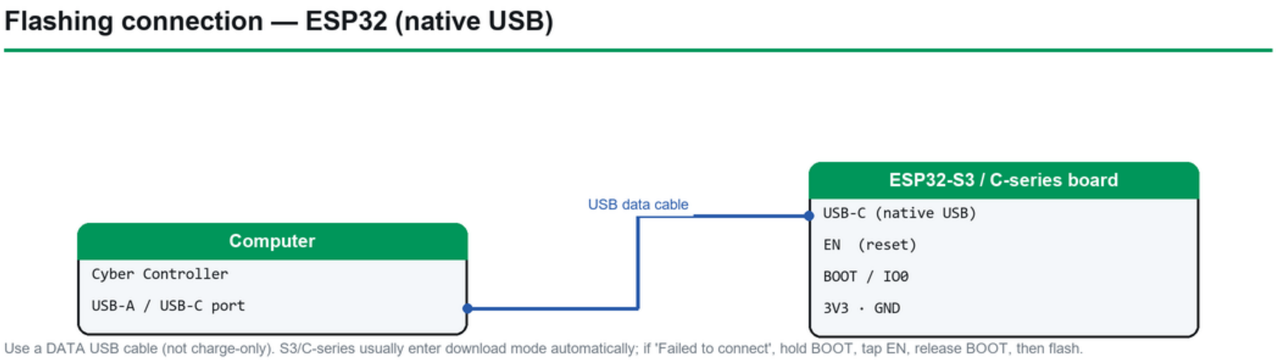
Avoid: ordering the plain **T-Deck** (no GPS, older variant) when you want the **Plus**, and avoid the wrong-band SKU. MCLite does **not** target generic Heltec/RAK MeshCore boards — for those, flash mainline MeshCore firmware instead (different profile/tooling).

4. Building / Assembly

Both targets are **fully assembled commercial devices — no soldering or wiring**:

- **Antenna:** screw the band-correct **SMA LoRa antenna** onto the T-Deck Plus before powering on. The T-Watch Ultra uses an internal antenna.
- **microSD:** format **FAT32 (≤ 32 GB)** and insert it. You can boot with a blank card (MCLite auto-generates an identity plus the **Public** and **#mclite** channels) or pre-load a `config.json` built with the Config Tool (see §5/§7).
- **USB-serial driver:** the ESP32-S3 uses **native USB-CDC** (no CP210x/CH340 chip on these boards), so modern Windows/macOS/Linux enumerate it automatically. If no port appears, it is almost always a **charge-only cable** — swap to a data cable.
- **Charge first:** give a new unit a short charge; a flat battery can make the board brown out during a USB flash on weak ports.
- **Per-board notes:** T-Deck Plus input is the **trackball + QWERTY**; T-Watch Ultra is **touch + on-screen keyboard**. SOS and lock gestures differ between them (see §7).

5. Flashing & First Run (via Cyber Controller)



How to connect the board to flash it via Cyber Controller (native-USB ESP32).

The `mclite` profile fetches the **latest GitHub release** asset, filters to the **`.bin`** file for your board (the release also ships a `config_tool.html`, which is excluded), and writes it as a **single merged image at offset `0x0`** via esptool — the same scheme the upstream web flasher uses.

1. Connect the board by **USB-C (data cable)**; open Cyber Controller → **Flash** tab.
2. **Port:** select the board's COM/tty. If missing, click *Refresh*; still missing → cable/driver issue (see §8).
3. **Firmware Profile:** `MCLite (MeshCore)` (profile id `mclite`).
4. **Board / variant:** choose the exact board — **LilyGo T-Deck Plus** or **LilyGo T-Watch Ultra**. The two ship **different merged `.bin` files**; flashing the wrong one will not run correctly.
5. Click **Flash.** Cyber Controller targets **esp32s3**, fetches the matching `.bin`, and writes it at **`0x0`** at **115200 baud** (flash mode `qio`, freq `80m`, 16 MB). Backend = esptool.
6. **First boot:** with a microSD inserted, MCLite auto-creates an identity + the **Public** and **#mclite** channels and is immediately usable. Set your **region preset** so the radio is legal: - EU/UK/CH → **869.618 MHz, SF8, BW 62.5 kHz, CR8, 22 dBm** - US/Canada → **910.525 MHz, SF7, BW 62.5 kHz, CR5, 22 dBm** - Australia (wide) → **915.800 MHz, SF10, BW 250 kHz, CR5, 22 dBm** (Victoria preset: 916.575 MHz) **Every member of your mesh must use identical radio settings** or they cannot hear each other.

> **Profile note:** the JSON marks the `0x0` offset as **"pending hardware verification (Stage-5 gate)."** If a flash boot-loops, **verify** the offset against the upstream web flasher / esptool command for your exact firmware version before retrying (see §9). The documented manual command is `esptool.py write_flash 0x0 mclite-<version>.bin`.

6. Integrate into Cyber Controller

- **Profile:** `mclite` — backend **esptool**, chip **esp32s3**, **merged single-bin** @ **`0x0`**, default baud **115200**, resolver `github_release` (latest release, `.bin` assets only). **Not suicide-capable**; no dangerous-command set.
- **Flash tab:** does all firmware writing (§5). Use **Backup** to dump the current flash before overwriting if you are replacing other firmware (e.g. Meshtastic) on the same board.
- **Devices tab:** select the port and **Connect** for a **serial console**. Note the profile's `protocol` is `null` — Cyber Controller provides a **generic serial terminal**, not a MeshCore-aware parser, so it shows boot/log output but does not decode the mesh into the Targets pool. This firmware is **not** meant to be driven by an attack command palette.
- **Where the real control lives — Companion mode** (one transport active at a time):
- **Bluetooth LE** → official **MeshCore iOS/Android** apps; first pairing shows a **6-digit PIN** (generated once and saved).
- **WiFi** → desktop/CLI via **`meshcore-cli`** (also `meshcore.js` / `meshcore_py`); listens on **port 5000** (no auth — keep it on a trusted network).
- **USB** → wired **USB-CDC**; the binary companion protocol mutes serial logging while active.
- *Constraint:* messages **composed on-device** do not sync back to the companion app (a MeshCore protocol limit); received and app-sent messages appear in both.
- **Config Tool:** build/edit `config.json` (identity, contacts, channels, radio params, language, themes) in the offline browser tool, then copy it to the SD card root — one person can configure a whole group and distribute the file.

7. Usage (end-to-end)

1. **Set region + identity:** apply the correct **region preset** (§5) and confirm a display name; ensure the **SD card** is inserted so history/config persist.
2. **Join/create channels:** start with **Public** / **#mclite**, or add **private** and **hashtag** channels; everyone in a channel shares its key/settings.
3. **Direct message:** pick a contact and send — DMs are **end-to-end encrypted** (Ed25519); the private key never leaves the device.
4. **Share location:** send a manual **GPS position** (lat/lon or MGRS) with a chosen **privacy grid** (coarsen to ~100 m up to ~50 km) so you don't broadcast an exact fix.
5. **Telemetry:** request a contact's **battery / position / distance** (subject to per-contact permissions); a **low-battery alert** auto-broadcasts at the threshold (default 15%, range 5–50%).
6. **SOS:** trigger the emergency alert — on the **T-Deck Plus** via a **trackball long-press (~6 s)**; on the **T-Watch Ultra** via the touch UI (verify exact gesture in-app).
7. **Room servers:** connect to a community **room server** (store-and-forward) to post to group boards and retrieve messages missed while offline.
8. **Map view:** load an **offline tile pack** onto the SD card to see contacts on a slippy map.
9. **Bridge to phone/CLI:** enable **companion mode** (§6) to chat from the MeshCore app over BLE, or from `meshcore-cli` over WiFi/USB.
10. **Lock when idle:** set a lock mode (none / key 1-s hold / **4–8-char PIN**), optionally auto-lock on display dim.

8. Troubleshooting

- **No COM port:** you're almost certainly on a **charge-only cable** — use a data cable. ESP32-S3 is native USB-CDC (no CP210x/CH340 needed). On Linux add your user to `dialout` and replug.
- **Flash fails / "Failed to connect":** hold **BOOT** while plugging in (or during connect) to force download mode, lower the flash baud in Settings, and close any serial monitor/companion session holding the port.
- **Boots but wrong/garbled UI:** you likely flashed the **other board's `.bin`** — re-flash selecting the exact board (T-Deck Plus vs T-Watch Ultra).
- **Boot-loop after flash: Erase Flash**, then re-flash. If it persists, **verify the `0x0` offset** against the upstream web flasher for your firmware version (profile offset is a Stage-5 gate — see §5/§9).
- **No SD features / settings don't persist:** card must be **FAT32, ≤ 32 GB**, inserted before boot; reformat if needed.
- **Can't reach other nodes: radio settings must match exactly** across the group — same region preset (frequency, SF, BW, CR) and channel keys. One mismatched parameter = silence.
- **Weak/no range or radio gets hot:** confirm the **band-correct antenna is attached** before TX (especially T-Deck Plus SMA); verify the board SKU matches your region's band.
- **Companion app won't pair:** BLE uses a **6-digit PIN** (shown once); only **one transport** (BLE *or* WiFi *or* USB) is active at a time — disable the others. WiFi companion is **port 5000, no auth** — use a trusted network.
- **On-device messages missing in app:** expected — on-device-composed messages don't sync to the companion app (MeshCore protocol limitation).

9. Sources

- Upstream firmware: https://github.com/laserir/MCLite (README, releases, web flasher, config tool).
- Web flasher: https://laserir.github.io/MCLite/tools/web-flasher/
- Config tool: https://laserir.github.io/MCLite/tools/config-tool/mclite_config_tool.html
- MeshCore project / protocol: https://meshcore.co.uk/, https://docs.meshcore.io/, https://github.com/meshcore-dev/MeshCore, flasher https://flasher.meshcore.io/.
- MeshCore vs Meshtastic background: Austin Mesh, NodakMesh, Seeed Studio comparison articles.
- Hardware specs: LilyGo store (https://lilygo.cc/) T-Deck Plus & T-Watch Ultra; CNX Software / Hackster / Micro Center / Amazon / Newegg / OpenELAB listings.
- Cyber Controller profile: `src/config/profiles/mclite.json` (esptool, esp32s3, merged @ `0x0`, 115200, 16 MB qio/80m; offset = Stage-5 verification gate).
- **Verify before relying on:** the `0x0` flash offset for your firmware version, your region's exact ISM-band frequency/power/duty-cycle limits, and the LilyGo board's band SKU at purchase time. Vendor links, prices, and presets change.